

ARMORIQ x OPENCLAW HACKATHON

The Context

Autonomous agents are no longer confined to chat interfaces.

With frameworks like OpenClaw, AI agents can operate directly on a user's local system. They can read and modify files, execute commands, interact with applications, manage development workflows, generate content, and perform multi step tasks without waiting for continuous prompts.

This unlocks a new level of capability.

It also introduces a new level of risk.

When agents operate inside real systems, mistakes are no longer hypothetical. They can overwrite code, publish unintended content, send messages, execute scripts, or make irreversible changes.

The challenge is no longer how to make agents smarter.

The challenge is how to make them trustworthy.

The Core Question

How do we design autonomous OpenClaw agents that can act independently while remaining strictly aligned with user defined intent?

An instruction such as:

"Clean this up."

"Handle this."

"Take care of it."

should not result in unintended execution.

Autonomy must not mean loss of control.

Your Challenge

Build an OpenClaw based autonomous system that:

- Performs meaningful multi step reasoning
- Executes real actions
- Enforces clear intent boundaries at runtime
- Demonstrates blocking of unauthorized behavior

The goal is not to build the most feature rich agent.

The goal is to demonstrate intent aware execution.

Your system should show:

- Clear separation between reasoning and execution
- Explicit validation of intent before action
- Deterministic enforcement of constraints
- Observable blocking when rules are violated

Delegation scenarios are optional but will receive bonus consideration.

Example Directions

These are illustrative, not restrictive.

- A developer assistant (a single agent or a group of agents) that modify code but
 - cannot touch protected files
 - can only touch allowed files
 - can not touch files outside a particular module
 - can prove that the changes made are within scope
- A content automation agent(s) that can generate assets but
 - cannot use certain words style
 - cannot publish outside approved platforms
- A system management assistant that can organize files but cannot access sensitive directories
 - cleans directories
 - reorganize directories
 - remove temporary files
 - monitor disk usage
- A physician agent that helps him with his day to day
 - hear patients-doctor conversations to transcribe them

- suggest potential diagnosis based on patients history
- suggests potential remedies prescriptions abiding
- A group of AI agents that mimic scientific peer review process that enforces on a set of policies on
 - **author agents**: come up with a new ideas and submit manuscripts
 - **chair agents**: instantiates reviewer agent with relevant expertise and assigns them paper
 - **reviewer agents**: review papers based on their expertise and give reviews along with a decision
- A legal assistant agent(s) that operate within legal limits to
 - take in cases and decide on its merits
 - prepares a plan of action, ensure client-attorney privilege
 - executes what it can do and outlines what it cannot
 - handles communication with other participants

Your system may target any domain, as long as it demonstrates intent aware enforcement.

RULE BOOK

Team Structure

- Teams of 2 to 4 members
- All implementation must be completed during the hackathon
- Pre built libraries and frameworks are allowed
- Fully pre built agent systems are not allowed

Technical Requirements

Every submission must include:

1. An OpenClaw based autonomous agent
2. Real execution of actions within a system
3. An intent validation layer before execution
4. Policy based runtime enforcement

Pure chatbot demos without execution will not qualify.

Architectural Expectations

Submissions must clearly demonstrate:

- Separation between reasoning and execution
- A visible enforcement layer
- Logging or traceability of decision making

The system must show:

- At least one allowed action
- At least one blocked action
- Clear reasoning for why the action was allowed or blocked

Intent and Policy Design

Each team must define:

- A structured intent model
- A policy model with enforceable constraints

Some example constraints:

- Directory scoped access
- Command restrictions
- Content type limitations
- Platform restrictions
- Time based restrictions
- Spend or usage limits

Hardcoded if else checks without structured intent validation will not be considered sufficient.

Delegation Bonus

Teams that implement bounded delegation will receive bonus points.

A valid delegation scenario must demonstrate:

- Limited scope authority
- Explicit constraints on the delegated agent
- Blocking of attempts to exceed granted authority

Delegation is not required, but high quality implementations will be rewarded.

Judging Criteria

Projects will be evaluated on:

A. Enforcement Strength

Are constraints technically enforced

Are violations deterministically blocked

B. Architectural Clarity

Is reasoning clearly separated from execution

Is the enforcement layer explicit and well designed

C. OpenClaw Integration

Does the system meaningfully leverage OpenClaw capabilities

D. Accurate Delegation Enforcement

If implemented, does the delegation mechanism correctly enforce scope boundaries

E. Use Case Depth

Is the scenario realistic and thoughtfully designed

Submission Requirements

Each team must submit:

1. Source code repository
2. Architecture diagram
3. Short document describing:
 - a. Intent model
 - b. Policy model
 - c. Enforcement mechanism
4. Three minute demo video demonstrating:
 - a. System overview
 - b. Allowed action
 - c. Blocked action
 - d. Explanation of enforcement

Finalist teams will present live with Q and A.